response is received, then a fallback behavior is executed, if any. Otherwise, an exception is thrown.

DEFINING A TIMEOUT FOR WEBCLIENT

Project Reactor provides a `timeout()` operator you can use to define a time limit for completing an operation. You can chain it with the result of the `WebClient` call to continue the reactive stream. Update the `getBookByIsbn()` method in the `BookClient` class as follows to define a timeout of 3 seconds.

Listing 8.23 Defining a timeout for the HTTP interaction

```
...
public Mono<Book> getBookByIsbn(String isbn) {
  return webClient
    .get()
    .uri(BOOKS_ROOT_API + isbn)
    .retrieve()
    .bodyToMono(Book.class)
    .timeout(Duration.ofSeconds(3));      ❶
}
...
```

❶ Sets a 3-second timeout for the GET request

Instead of throwing an exception when the timeout expires, you have the chance to provide a fallback behavior. Considering that Order Service can't accept an order if the book's availability is not verified, you might consider returning an empty result so that the order will be rejected. You can define a reactive empty result using `Mono.empty()`. Update the `getBookByIsbn()` method in the `BookClient` class as follows.

Listing 8.24 Defining timeout and fallback for the HTTP interaction

```
...
public Mono<Book> getBookByIsbn(String isbn) {
```